# *ML24/25-03 Implement Anomaly Detection Sample*

Mahbubur Rahman
mahbubur.rahman@stud.fra-uas.de

Md Rakibul Islam
rakibul.islam@stud.fra-uas.de

Md Zihadul Islam Joni
zihadul.joni@stud.fra-uas.de

***Abstract*— HTM (Hierarchical Temporal Memory) is a machine learning algorithm, biologically inspired, both structurally and functionally, by the neocortex of a human brain, which uses a hierarchical network of nodes to process time-series data in a distributed way. Each node, or column, can be trained to learn and recognize patterns in input data. This can be used to process information, recognize, and identify patterns, and make future predictions based on past learning. It is a promising approach for anomaly detection and prediction in a variety of applications in different domains. We have implemented an anomaly detection sample using HTM, such that it learns multiple simple numeric integer sequences given to the HTM model as input and tries to learn patterns in it. It will then try to identify anomalies by comparing the real data with the predicted data from learning, with a certain threshold of tolerance.**

***Keywords—HTM, anomaly detection, machine learning, multi-sequence learning.***

## I. INTRODUCTION

The Hierarchical Temporal Memory (HTM) algorithm is a machine learning algorithm based on the core principles of the Thousand Brains Theory [1]. It takes its inspiration from the structure and function of the neocortex- a large, sophisticated, and complex part of the human brain. The neocortex is believed to be responsible for processing sensory information, cognition, and storing and retrieval of memory.

HTM tries to imitate the same fundamental working process of the neocortex by learning complex temporal patterns and relationships in data and making future predictions from it [2]. It is particularly suited for sequence learning modeling, similar to RNN methods like Long short-term memory (LSTM) and Gated recurrent unit (GRU) [3]. Hierarchy in HTM refers to the layered structure of a neural network. HTM networks consist of multiple layers of neurons. Each layer performs a specific type of computation, and information is passed on from lower layers to higher layers for processing it further. The lower layers receive input from the environment, like sensory data. The input is encoded into a distributed representation in these layers. Distributed representation is a way of representing information where only a few parts are active to indicate the presence of some features. The higher layers assimilate information from lower layers to form complex representations. This allows the network to identify more complex and abstract patterns in input data.

The Hierarchical Temporal Memory Cortical Learning Algorithm (HTM CLA) is a machine learning algorithm that is built in a way that tries to simulate the way the neocortex processes information in the human brain [4].

In Hierarchical Temporal Memory (HTM), the Scalar Encoder is responsible for converting numeric values into a Sparse Distributed Representation (SDR), which HTM can process. It ensures that similar values produce overlapping SDRs, allowing the system to recognize patterns and make predictions effectively. The encoder divides the input range into discrete buckets, where each value is mapped to a specific set of active bits within a fixed-length SDR. This design makes HTM robust to noise and allows for generalization across similar inputs. Additionally, the Scalar Encoder can handle cyclic data, such as time-of-day values, by ensuring that the representations wrap around correctly. This encoding mechanism is widely used in HTM for tasks like anomaly detection and sequence learning, as it efficiently transforms raw numerical data into a format suitable for HTM's learning algorithms [5].

In Hierarchical Temporal Memory (HTM), the Spatial Pooler is a core component responsible for converting raw input data into a stable Sparse Distributed Representation (SDR). It ensures that similar inputs produce similar sparse patterns while maintaining robustness to noise and minor variations. The Spatial Pooler operates through a competitive learning process where columns of neurons compete to become active based on input overlap. It enforces sparsity by activating only a small percentage of columns, ensuring efficient and noise-resistant encoding. Additionally, it adapts over time by strengthening connections to frequently occurring input patterns, enhancing the system's ability to generalize and recognize familiar data. This process helps HTM learn and represent spatial patterns effectively, making it a crucial step before temporal learning in sequence modeling and anomaly detection [6].

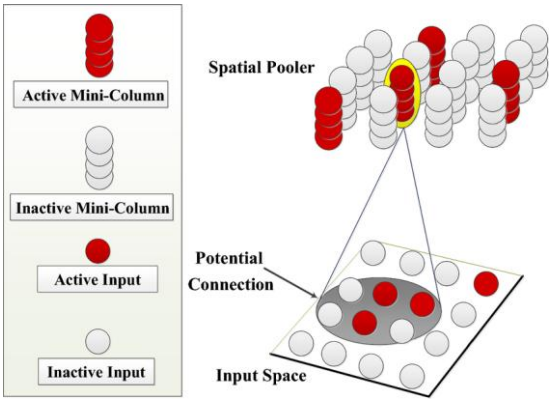


Figure 2: Temporal Memory Spatial Pooler

Multisequence learning in Hierarchical Temporal Memory (HTM) refers to the system's ability to simultaneously learn and recognize multiple distinct temporal sequences. HTM processes input data through Spatial Poolers, which encode the data into Sparse Distributed Representations (SDRs), and Temporal Memory, which learns the temporal transitions between these representations. When learning multiple sequences, HTM handles each sequence independently, ensuring that different patterns or events occurring over time are learned in parallel. The system uses its temporal columns to track and predict sequence transitions, making it capable of distinguishing between overlapping sequences. Each sequence's transitions are learned within the context of its own unique pattern, which allows HTM to model complex time-series data. The Temporal Memory component maintains a detailed record of the sequence order, enabling the prediction of future steps. The system's ability to handle multiple sequences simultaneously enhances its generalization capabilities, making it effective for anomaly detection and pattern recognition. HTM can adapt to new sequences and differentiate between them even when they share similarities. This multisequence learning capability is particularly useful in applications like time-series forecasting, real-time data analysis, and detecting anomalous behaviors in various domains. Overall, HTM's structure enables efficient learning and prediction of multiple overlapping sequences in a dynamic environment [7].

HTM CLA consists of a few major components for processing input information. In the beginning, the raw input data is encoded by the encoder. It helps to convert the raw input data into sparse distributed representation (SDR), which is then fed to a spatial pooler. SDR is a way of representing information in binary format using a small number of active bits [8]. This allows for efficiency in the storage, processing, and robustness of data. The spatial pooler creates a new SDR from the output from the encoder and converts it into SDR more robust to noise. This output is then processed by the Temporal Memory component. It is responsible for recognizing and learning patterns in data. The Classifier component then classifies the input data based on patterns it has learned previously. It also makes prediction patterns based on learned patterns and the input data. The component Homeostatic Plasticity Controller ensures that new patterns are learned by the system over time. Figure 2 shows how input data is processed in an HTM system.

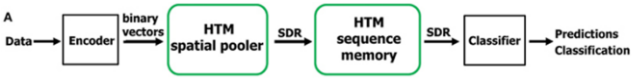


Figure 1: HTM pipeline.

## II. Methods

We are going to use Neocortex API, which is based on HTM CLA, for implementing our sample project in the C#/.NET framework. For training and testing our project, we are going to use artificially generated data, which contains numerous samples of simple integer sequences in the form of (1,2, 3,). These sequences will be placed in a few comma-separated value (CSV) files. There will be two folders inside our main project folder, training (or learning) and predicting. These folders will contain a few of these CSV files. Predicting folders contain data like training, but with added anomalies randomly added inside it. We are going to read data from both folders and train our HTM model using it. After that, we are going to take a part of the numerical sequence, trim it in the beginning, from all the numeric sequences of the predicting data and use it to predict anomalies in our data which we have placed earlier, and this will be automatically done, without user interaction.

As artificially generated data, we are going to take the network traffic load (in percentage, rounded off to the nearest integers) of a sample web server. The values of this load, taken overtime, are represented as numerical sequences. For testing our sample project, we will consider the values inside [65,75] as normal values, and anything outside it to be anomalies. Our predicting data comprises of anomalies between values between [0, 100] placed at random indexes. Combined data from both training and predicting folder. After learning the sequence, testing sequence is used to detect anomalies. in Figure 2. We can see the actual and predicting predicting sequences with anomalies, more files can be found [9] .

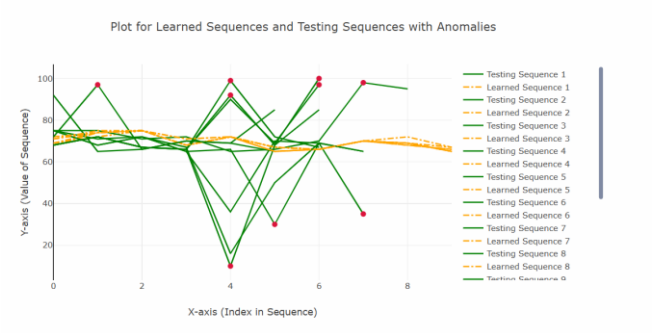


Figure 2: Actual Predicting Sequences with Anomalies.

**A. HTM Configuration:** We are going to use multisequencelearning class of Neocortex API as a base of our project. It will help us with both training our HTM model and using it for prediction. The class works in the following way:

a. HTM Configuration is taken and memory of connections is initialized. After that, the HTM Classifier, Cortex layer, and HomeostaticPlasticityController are initialized.

b. After that, the Spatial Pooler and Temporal Memory is initialized.

c. After that, spatial pooler memory is added to the cortex layer and trained for a maximum number of cycles.

d. After that, temporal memory is added to the cortex layer to learn all the input sequences.

e. Finally, the trained cortex layer and HTM classifier are returned.

Encoder and HTM Configuration settings are needed to be passed to relevant components in this class. We are going to use the classifier object from trained HTM to predict value, which will be eventually used for anomaly detection.

We are going to train and test data between the range of integer values between 0 and 100 with no periodicity, so we are using the following settings given in listing 1. We are taking 21 active bits for representation. There are 101 values which represent integers between [0, 100]. We are calculating our input bits using n = buckets + w – 1 = 101+21-1 = 121. [3]

```
int inputBits = 121;
int numColumns = 1210;
.....……
Dictionary<string, object> settings = new
Dictionary<string, object>()
        {
           { "W", 21},
           { "N", inputBits},
        };
```

Listing 1: Encoder settings for our project

Minimum and maximum values are set to 0 and 100 respectively, as we are expecting all the values to be in this range only. In other cases, these values must be changed depending on the input data. We have made no changes to the default HTM Config.

**B. Execution Steps:** Our project is executed in the following steps-

a. To train our HTM Engine, we used the MultiSequenceLearning class in the NeoCortex API. Firstly, we will read and train the HTM Engine using the data from both our training (learning) and predicting (predictive) folders, which are present as numerical sequences in CSV files in the 'training' and 'predicting' folders inside the project directory. We will read numerical sequence data from the prediction folder for testing purposes, remove the first few elements (thus effectively turning the data into a subsequence of the original sequence; we have already inserted anomalies at random indexes into this data), and then use it to detect anomalies. Please take note that all files inside the folders are read with the.csv extension, and exception handlers are set up in case the file format is incorrect. We are employing artificial integer sequence data of network load for this project, which is saved inside of CSV files and is rounded off to the nearest integer, in percentage. Example of a csv file within training folder.

```
69,72,75,68,72,67,66,70,72,67
69,72,75,68,72,67,66,70,69,67
68,74,75,68,72,67,66,70,69,65
71,74,75,68,72,67,66,70,69,65
```

Listing 2: Dataset 1

Normally, the values stay within the range of 65 to 75. All values outside of this range are considered anomalies for testing purposes. However, to identify anomalies, we have a csv file in the predicting folder. Typically, some of the data in this file does not fall within 65 and 75.

```
71,74,98,68,92,65,66,70,69,65
71,74,75,68,72,65,66,30,69,35
71,74,75,71,72,65,36,70,69,65
71,75,75,71,72,65,66,70,98,95
```

Listing 3: Dataset 2

b. In the beginning, we have ExtractSequencesFromFolder method of CsvSequenceFolder class to read all the files placed inside a folder. These classes keep track of the read sequences in a list of numerical sequences that will be used repeatedly in the future. To handle non-numeric data, some classes have incorporated exception handling inside. With the Trimsequences technique, data can be trimmed. It returns a numeric sequence after trimming one to four components (numbers 1 through 4) from the start. Both the methods are given in listing 2.

```
public List<List<double>>
ExtractSequencesFromFolder()
       {
        ....
         return folderSequences;
       }

public static List<List<double>>
TrimSequences(List<List<double>> sequences)
       {
       ....
         return trimmedSequences;
       }
```

Listing 4: Important methods in CsvSequenceFolder class

c. After that, the method ConvertToHTMInput of CSVToHTMInputConverter class is there which converts all the read sequences to a format suitable for HTM training. It is shown in listing 3.

```
Dictionary<string, List<double>> dictionary = new
Dictionary<string, List<double>>();
for (int i = 0; i < sequences.Count; i++)
   {
    // Unique key created and added to dictionary for
HTM Input
    string key = "S" + (i + 1);
    List<double> value = sequences[i];
    dictionary.Add(key, value);
   }
    return dictionary;
```

Listing 5: ConvertToHTMInput method

d. After that, we have ExecuteHTMModelTraining method of HTMTrainingManager class to train our model using the converted sequences. The numerical data sequences from training (for learning) and predicting folders are combined before training the HTM engine. This class returns our trained model object predictor.

```
.....
MultiSequenceLearning learning = new
MultiSequenceLearning();
predictor = learning.Run(htmInput);
.....
.....
List<List<double>> combinedSequences = new
List<List<double>>(sequences1);
combinedSequences.AddRange(sequences2);
.....
```

Listing 6: Code demonstrating how data is passed to HTM model using instance of class multisequence learning

e. We will use HTMAnomalyExperiment class to detect anomalies. This class works in the following way, We pass on the paths of the training (learning) and predicting folder to the constructor of this class. The Run method encompasses all the important steps that we are going to help run this project from the beginning.

1. At first, we start our model training using the HTMTrainingManager class by passing paths of the training and predicting folder path using the constructor.

2. After that, we use CsvSequenceReader class to read our test data from predicting folder. Before starting our prediction, we use TrimSequences method of this class to trim a few elements in the front before testing, as shown in listing 5. This method trims between 1 to 4 elements of a sequence. The number between 1 to 4 is decided randomly, and it essentially returns a subsequence. We will use this data for predicting anomalies. Please note that the data read from predicting folder contains anomalies at random indexes in different sequences.

```
.....
CsvSequenceFolder testSequencesReader = new
CsvSequenceFolder(_predictingCSVFolderPath);
var inputSequences =
testSequencesReader.ExtractSequencesFromFolder();
var trimmedInputSequences =
CsvSequenceFolder.TrimSequences(inputSequences);
.....
```

Listing 7: Trimming sequences in testing data

Path to training and predicting folder is set as default and passed on the constructor, or can be set inside the class manually.

```
.....
_trainingCSVFolderPath =
Path.Combine(projectBaseDirectory, trainingFolderPath);
_predictingCSVFolderPath =
Path.Combine(projectBaseDirectory,
predictingFolderPath);
.....
```

Listing 8: Path to training and predicting folder

We have used multisequencelearning class file's RunExperiment method to train the dataset. Method provides an example of how multisequence learning functions. As a summary, Initialization of connection memory and HTM configuration are performed. The HTM Classifier, Cortex layer, and Homeostatic Plasticity Controller are then initialized. Following that, Temporal Memory and Spatial Pooler are initialized.

```
.....
TemporalMemory tm = new TemporalMemory();
SpatialPoolerMT sp = new SpatialPoolerMT(hpc);
.....
```

Listing 9: Temporal Memory and Spatial Pooler

The cortical layer is then added with spatial pooler memory, which is trained for the maximum number of cycles.

```
.....
layer1.HtmModules.Add("sp", sp);
int maxCycles = 3500;
for (int i = 0; i < maxCycles && isInStableState == false;
i++)
.....
```

Listing 10: Trained max cycles.

In order to learn every input sequence, temporal memory is then introduced to the cortical layer.

```
.....
layer1.HtmModules.Add("tm", tm);
foreach (var sequenceKeyPair in sequences){
.....
}
.....
```

Listing 11: temporal memory and the cortical layer.

The HTM classifier and trained cortical layer are finally returned. The AnomalyGraphs class is used to plot graphs of sequences of data and their anomalies. The class contains one static method, PlotGraphWithAnomalies, which takes three parameters: a list of arrays of doubles, where each array represents a sequence of data, and all AnomalyIndices: a list of lists of integers, where each list represents the indices of the anomalies in a sequence.

```
public static void
CompareGraphForSequences(List<double[]>
allLearnedData, List<double[]> allTestingData)
```

Listing 12: Anomalies in a sequence

The method works by creating a line graph for both training and learning sequences and a scatter plot for its anomalies. It then combines all the graphs into a chart and displays it. This method uses the XPlot.Plotly library to create the graphs and the chart. A Scatter object graph is created for the sequence. The x-values are the indices of the data points, and the y-values are the data points themselves. The graph is a line graph and is named "Sequence" followed by the index of the sequence.

```
var graph = new Scatter
{
   x = Enumerable.Range(0, data.Length).ToArray(),
   y = data,
   mode = "lines",
   name = "Sequence" + i
};
```

Listing 13: Line graph index of the sequence.

A Chart object chart is created by plotting all the graphs in allGraphs and allAnomalies. Finally, we saved the chart in html format here and also chart is displayed.

```
var chart = Chart.Plot(allGraphs.Concat(allAnomalies));
File.WriteAllText(filePath, chart.GetHtml());
chart.Show();
```

Listing 14: html format chart.

3. After that we pass on each sequence of the test data one by one to DetectAnomaly method. DetectAnomaly method is the method responsible for anomaly predictions in our data as shown in listing 7. We also placed an exception handling to handle non-numeric data, or if a testing sequence is too small (below 2 elements).

```
foreach (List<double> list in triminputtestseq)
        {
           double[] lst = list.ToArray();
           try
           {
              DetectAnomaly(myPredictor, lst);
           }
           catch (ArgumentException ex)
           {
              Console.WriteLine($"Exception caught:
{ex.Message}");
           }
        }
```

Listing 15: Passing of testing data sequences to DetectAnomaly method

Exception handling is present, such that errors thrown from Detect Anomaly method can be handled (like passing of non-numeric values, or the number of elements in the list of less than two).

e. Detect Anomaly method is the most important part of the code in this project. We use this to detect anomalies in our test data using our trained HTM model.

This method traverses each value of the tested sequence one by one in a sliding window manner and uses a trained model predictor to predict the next element for comparison. We use an anomaly score to quantify the comparison, by taking the absolute value of the difference between the predicted value and real value. If the prediction (absolute difference ratio) crosses a certain tolerance level (threshold value), preset to 10%, it is declared as an anomaly and outputted to the user.

In our sliding window approach, naturally, the first element is skipped, so we ensure that the first element is checked for anomaly in the beginning. So, in the beginning, we use the second element of the list to predict and compare the previous element (which is the first element). A flag is set to control the command execution; if the first element has an anomaly, then we will not use it to detect our second element. We will directly start from the second element. Otherwise, we will start from the first element as usual.

When we traverse the list one by one to the right, we pass the value to the predictor to get the next value and compare the prediction with the actual value. If there's anomaly, then it is outputted to the user, and the anomalous element is skipped. Upon reaching the last element, we can end our traversal and move on to the next list.

Detect Anomaly is the main method from the ExtractSequencesFromFolder class which detects anomalies in our data. It traverses each value of a list one by one in a sliding window manner and uses a trained model predictor to predict the next element for comparison. We use an anomaly score to quantify the comparison and detect anomalies; if the prediction crosses a certain tolerance level, it is declared as an anomaly. In our sliding window approach, naturally, the first element is skipped, so we ensure that the first element is checked for anomaly in the beginning. We can get our prediction in a list of results in format of "NeoCortexApi.Classifiers.ClassifierResult`1[System.String] " from our trained model Predictor as shown in Listing 8.

```
var res = predictor.Predict(item);
```

Listing 16: Using trained model to predict data

Here, assume that item passed to the model is of int type with value 8. We can use this to analyze how prediction works. When this is executed,

```
//Input
foreach (var pred in res)
 {
   Console.WriteLine($"{pred.PredictedInput} -
{pred.Similarity}");
    }

//Output

S2_2-9-10-7-11-8-1 - 100
S1_1-2-3-4-2-5-0 - 5
S1_-1.0-0-1-2-3-4 - 0
S1_-1.0-0-1-2-3-4-2 - 0
```

Listing 17: Accessing predicted data from trained model

We know that the item we passed here is 8. The first line gives us the best prediction with similarity accuracy. We can easily get the predicted value which will come after 8 (here, it is 1), and previous value (11, in this case). We use basic string operations to get our required values.

When we iteratively pass values to DetectAnomaly method using our sliding window approach, we will not be able to detect anomaly in the first element. So, in the beginning, we use the second element of the list to predict and compare the previous element (which is the first element). A flag is set to control the command execution; if the first element has an anomaly, then we will not use it to detect our second element. We will directly start from the second element. Otherwise, we will start from the first element as usual.

Now, when we traverse the list one by one to the right, we pass the value to the predictor to get the next value and compare the prediction with the actual value. If there's an anomaly, then it is outputted to the user, and the anomalous element is skipped. Upon reaching the last element, we can end our traversal and move on to the next list.

We use anomaly score (difference ratio) for comparison with our already preset threshold. When it exceeds, probable anomalies are found. To run this project, use the following class/methods given in [Program.cs].

```
HTMAnomalyExperiment tester = new
HTMAnomalyExperiment();
tester.ExecuteExperiment();
```

Listing 18: using program.cs class methods

**Accuracy Calculation:**

We have calculated accuracy and average accuracy in our HTMAnomalyDetector class. In our code, accuracy is based on the similarity score returned by the HTM predictor. Each prediction by the HTM model includes:

```
double similarity = predictionResult.First().Similarity;
```

Listing 19: Prediction by the HTM Predictor

This Similarity value percentage from 0 to 100 represents how close the predicted value is to the actual one — as interpreted by the HTM model. Inside this loop in the ProcessSequence method accuracy calculated:

```
for (int i = 0; i < sequence.Length - 1; i++)
{
   var predictionResult = predictor.Predict(sequence[i]);
   ...
   double similarity = predictionResult.First().Similarity;
   ...
   sequenceAccuracy += similarity;
}
```

Listing 20: Accuracy calculation

So, for each input value in the sequence, the HTM model makes a prediction and outputs a similarity score. These scores are summed up in sequenceAccuracy. After processing the full sequence, we calculated the average similarity of accuracy like this:

```
double avgAccuracy = sequenceAccuracy /
(sequence.Length - 1);
```

Listing 21: average similarity

This gives us the average similarity score for that sequence essentially our average accuracy of prediction for the whole sequence. We also maintain a global accuracy across all sequences:

```
_cumulativeAccuracy += avgAccuracy;
_sequenceCount++;
```

Listing 22: Global accuracy

Then later we calculated the overall average accuracy across all test sequences.

```
StoredOutputValues.totalAvgAccuracy =
_cumulativeAccuracy / _sequenceCount;
```

Listing 23: overall average accuracy

Upon completion of the experiment, the anomaly detection results are displayed on the screen, along with the HTM accuracy for each individual number sequence and the overall HTM accuracy for the entire experiment. Once the experiment is finished, a plotted graph automatically opens in the default browser, highlighting anomalies in the numerical sequence data from the predicting folder with red dots.

```
public static void
CreateGraphForSequences(List<double[]>
allLearnedData, List<double[]> allTestingData)
.....
allGraphs.Add(actualGraph);
allGraphs.Add(learnedGraph);
.....
var chart = Chart.Plot(allGraphs);
```

Listing 24: Code for generating plot.

## III. Results

Let us discuss the output of this experiment. For a brief analysis, we are going to discuss a part of our output next. If the sequence passed to our trained HTM engine is [72, 67, 66, 90, 69, 97], we get the following output with respective accuracies. As mentioned earlier, we have considered values within range of [65,75] to be non-anomalous. Anything outside of this range is considered an anomaly. Then we can analyze our output data and calculate the accuracy [10].

After the project's execution, we obtained the following [11].

We trained the model using 20 sequences and evaluated its accuracy in detecting anomalies within the test sequences provided table below.

| Index | Testing Sequence | Learned Sequences | Tolerance Value | Avg. Accuracy |
|---|---|---|---|---|
| 1 | {71,74,98,68,92,65,66,70,69,65} | Sequence | 0.2 | 28.77 % |
| 2 | {71,74,75,68,72,65,66,30,69,35} | Sequence | 0.2 | 25.67 % |
| 3 | {71,74,75,71,72,65,36,70,69,65} | Sequence | 0.2 | 16.02 % |
| 4 | {71,75,75,71,72,65,66,70,98,95} | Sequence | 0.2 | 35.12 % |
| 5 | {69,72,75,68,72,67,66,99,72,67} | Sequence | 0.2 | 41.90 % |
| 6 | {69,72,75,68,72,67,66,90,69,97} | Sequence | 0.2 | 63.19 % |
| 7 | {69,74,75,68,72,67,66,92,68,100} | Sequence | 0.2 | 41.86 % |
| 8 | {69,74,75,68,72,67,66,10,68,85} | Sequence | 0.2 | 50.10 % |
| 9 | {68,74,75,68,72,67,16,50,69,65} | Sequence | 0.2 | 39.29 % |
| 10 | {71,74,75,68,72,97,66,70,69,85} | Sequence | 0.2 | 33.45 % |

Table 1: Anomaly Detection Accuracy in Test Sequences

As we can see, the accuracy rate ranges from around 20% to 70%. In an anomaly detection algorithm, a high degree of accuracy on the sequence is desired. Our machine's hardware specifications prevent us from running programs with a lot of cycles and sequences. Nevertheless, by executing more data sequence and cycle, accuracy can be increased.

We have implemented grpahical view for learned sequence and testing sequence with anomaly using plots functionality can be found in AnomalyGraphs [12].

All the output result plots can be found in html format [13].

We generated 2 kinds of plots

1. Actual and Predicting sequence
2. Actual and Predicting sequence with anomalies

## IV. Discussion

Accuracy rate indicates the ratio of missed anomalies (actual anomalies that are classified as normal). Having a lower accuracy rate in our model is not desirable at all, and might be consequential in many cases, for example- fraud detection in financial transactions, etc. Generally, a model should have high accuracy. Having a high accuracy rate is also desirable but not essential, increased accuracy rate, like when normal cases are marked as anomalies, can lead to unnecessary investigation of anomalous data and wasted effort. A lower value of both makes a good model for anomaly detection.

HTM is generally well suited for anomaly detection, because it is able to detect anomalies in real-time stream of data, without needing to use data for training. It is also robust to noise in input data.

In this experiment, we can see that the accuracy rate is 20 to 70 percent, but we have tested our sample project in our local machine with less number of numerical sequences due to high computational resource requirements and time constraints. This can be improved if high computation resources are used and more time is used for training the model in other platforms, like the cloud. The results can be improved if we can use more amount of data and tune the hyper-parameters of our HTM model for the best performance suited for our input data.

## V. REFERENCES


[1] Hole, Kjell. (2016). The HTM Learning Algorithm. 10.1007/978-3-319-30070-2_11.

[2] Bamaqa, Amna & Sedky, Mohamed & Bosakowski, Tomasz & Bakhtiari Bastaki, Benhur. (2020). Anomaly Detection Using Hierarchical Temporal Memory (HTM) in Crowd Management. 10.1145/3416921.3416940.

[3] https://doi.org/10.1016/j.neucom.2018.09.098.

[4] Price, Ryan William, "Hierarchical Temporal Memory Cortical Learning Algorithm for Pattern Recognition on Multi-core Architectures" (2011). *Dissertations and Theses.* Paper 202. https://doi.org/10.15760/etd.202.

[5] https://studylib.net/doc/18531977/encoding-data-for-htm-systems.

[6] https://scholarworks.boisestate.edu/td/1486/.

[7] https://nsuworks.nova.edu/gscis_etd/1123/.

[8] https://www.researchgate.net/publication/309778443_A_comparative_study_of_HTM_and_other_neural_network_models_for_online_sequence_learning_with_streaming_data.

[9] https://github.com/mahbubur-r/neocortexapi/tree/Team_Anomaly_Detection/source/MySEProject/AnomalyDetectionSample/result/plots.

[10] https://github.com/mahbubur-r/neocortexapi/blob/46d0e63232643ce6e1758bb77fa0e73894f3fafd/source/MySEProject/AnomalyDetectionSample/HTMAnomalyDetector.cs#L127-L152

[11] https://github.com/mahbubur-r/neocortexapi/tree/Team_Anomaly_Detection/source/MySEProject/AnomalyDetectionSample/output

[12] https://github.com/mahbubur-r/neocortexapi/blob/Team_Anomaly_Detection/source/MySEProject/AnomalyDetectionSample/AnomalyGraphs.cs

[13] https://github.com/mahbubur-r/neocortexapi/tree/Team_Anomaly_Detection/source/MySEProject/AnomalyDetectionSample/result/plots